PlutoHacks 2026

VoidWatch

Fearless React-2-Attack Tool

prompt-injection detection and honeypot-monitoring demo

- agency for everyone, enabling everyday people against cyber-attacks -

Our LLM judge reads a piece of untrusted content (email, webpage, file text) and, treating everything inside it as data rather than instructions, scores how likely it is to contain a prompt injection attempt aimed at manipulating the AI agent.

Implementing our LLM Judge

- Choose a judge/provider and decide whether checked text may be sent to it.
- Add configuration for credentials/model via environment variables; never commit secrets.
- Write a judge module with a prompt that treats submitted content strictly as data and asks for structured JSON.
- Call it only for ambiguous heuristic scores (for example, 20–80) to reduce latency and cost.
- Validate the response—score range, allowed technique, and required fields; handle provider errors explicitly.
- Combine the judge score with the heuristic score, then apply the existing allow/deceive/block thresholds.
- Add tests for the prompt, response validation, threshold routing, and provider failures without requiring live API calls.
- Document setup, privacy, and limitations; add a Devpost “Built with” tag for the provider only after integration.

how the judge thinks and chooses…

- Detection layer: the shield
- Decision layer: a simple confidence threshold (below X → block, above X but not certain → deceive)
- Deception layer: one fake "inbox" with fake sensitive files and a fake external endpoint your own server controls; when the agent tries to act on an injection, it operates inside this fake world instead of the real one
- Dashboard: show real attack → shield catches it → attack gets redirected into the honeypot → here's what it tried to do, here's the fingerprint\

Build Order with Hackathons in Mind:

Hint: Document, document, document!

a. Heuristic layer only, wired to block/allow (get a working demo fast)

b. Add the LLM judge for the ambiguous middle range

c. Add one honeypot context and the redirect logic

d. Build the dashboard last

TIPS FOR HACKATHON BUILDERS:

Only call the LLM judge if the heuristics are ambiguous (say, 20–80 range), to keep latency and cost down for your demo.

VoidWatch's LLM judge uses Qwen 3.5 4B, a neural causal language model. In simple terms, it reads the text and generates a response one token at a time—in our case, a JSON classification with a risk score, technique, and explanation.

Under the hood, Qwen 3.5 uses a hybrid transformer architecture, combining Gated DeltaNet layers with attention layers. VoidWatch doesn't train or fine-tune the model; it gives it a security-classification prompt and asks it to judge only middle-range heuristic scores.

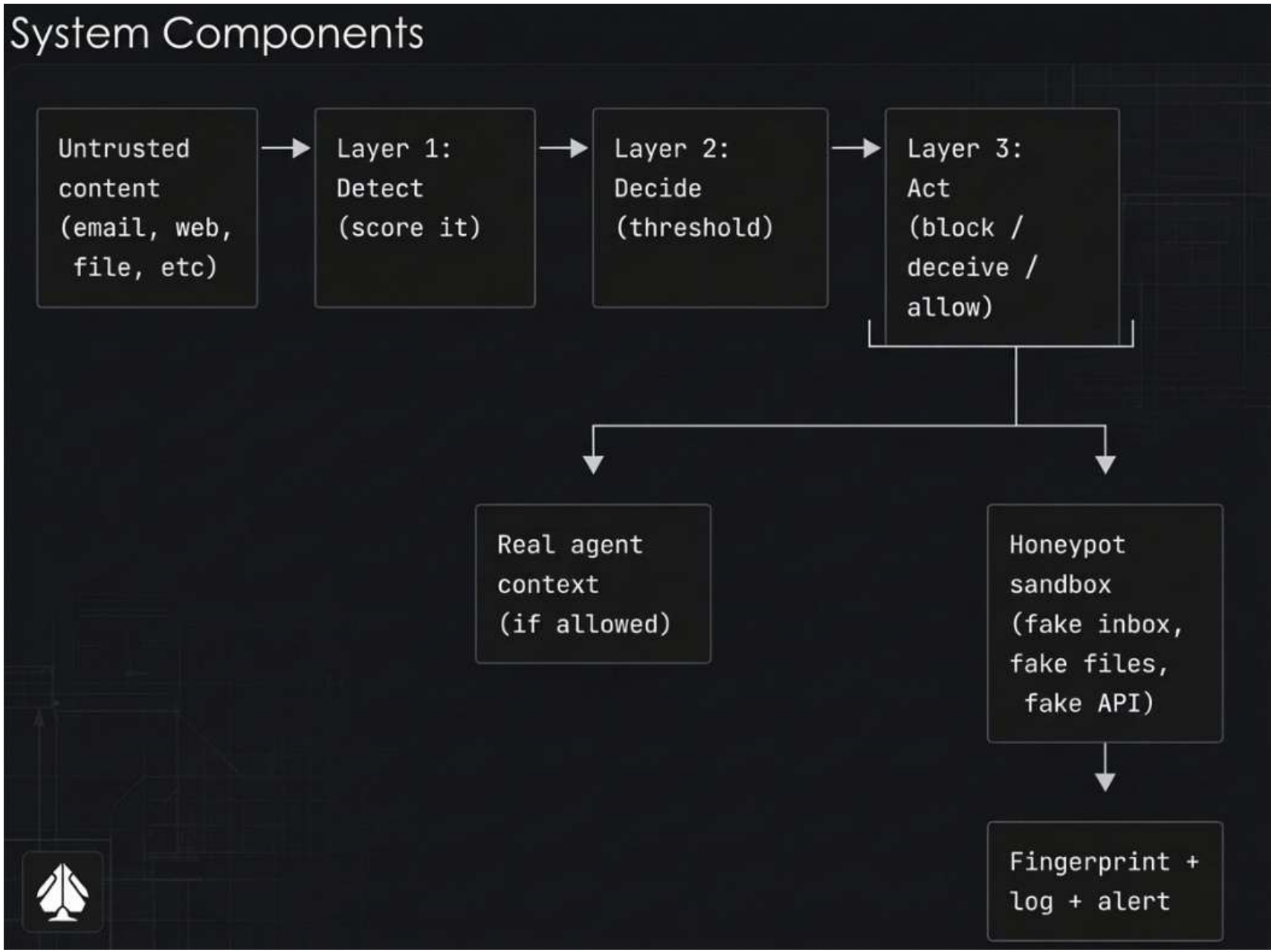
So, the project combines hand-written scoring rules with prompt-based AI classification.

In VoidWatch, we use Qwen 3.5 4B as a text classifier: we prompt it to label submitted content with an injection score, technique, and short reason. More precisely, Qwen itself is a generative language model being used for classification; it isn't a separately trained classification-only model.

Data flow, step by step…

1. Agent is about to read a piece of untrusted content (an email body, a scraped webpage, a PDF's text).
2. Before that content reaches the real LLM context, it's piped through the shield.
3. Shield returns a **risk score (0–100)** plus a **reason**.
4. Decision layer applies thresholds (tune these live during your demo):
    - **0–30**: allow, pass to real agent normally
    - **31–70**: suspicious, not certain → route to the **honeypot** instead of the real environment, let the agent "act" there
    - **71–100**: high confidence attack → block outright, log, alert, skip even the honeypot (no point feeding it a known pattern)
5. If routed to the honeypot: the agent is handed a parallel fake context (fake inbox, fake filenames, a fake "internal API" that the shield itself controls) and allowed to act freely.
6. Whatever the agent tries to do in the honeypot (send data to a URL, "delete" a file, forward an "email") gets captured and turned into a fingerprint: pattern used, payload style, target domain if any.
7. Dashboard shows the whole trail in real time.

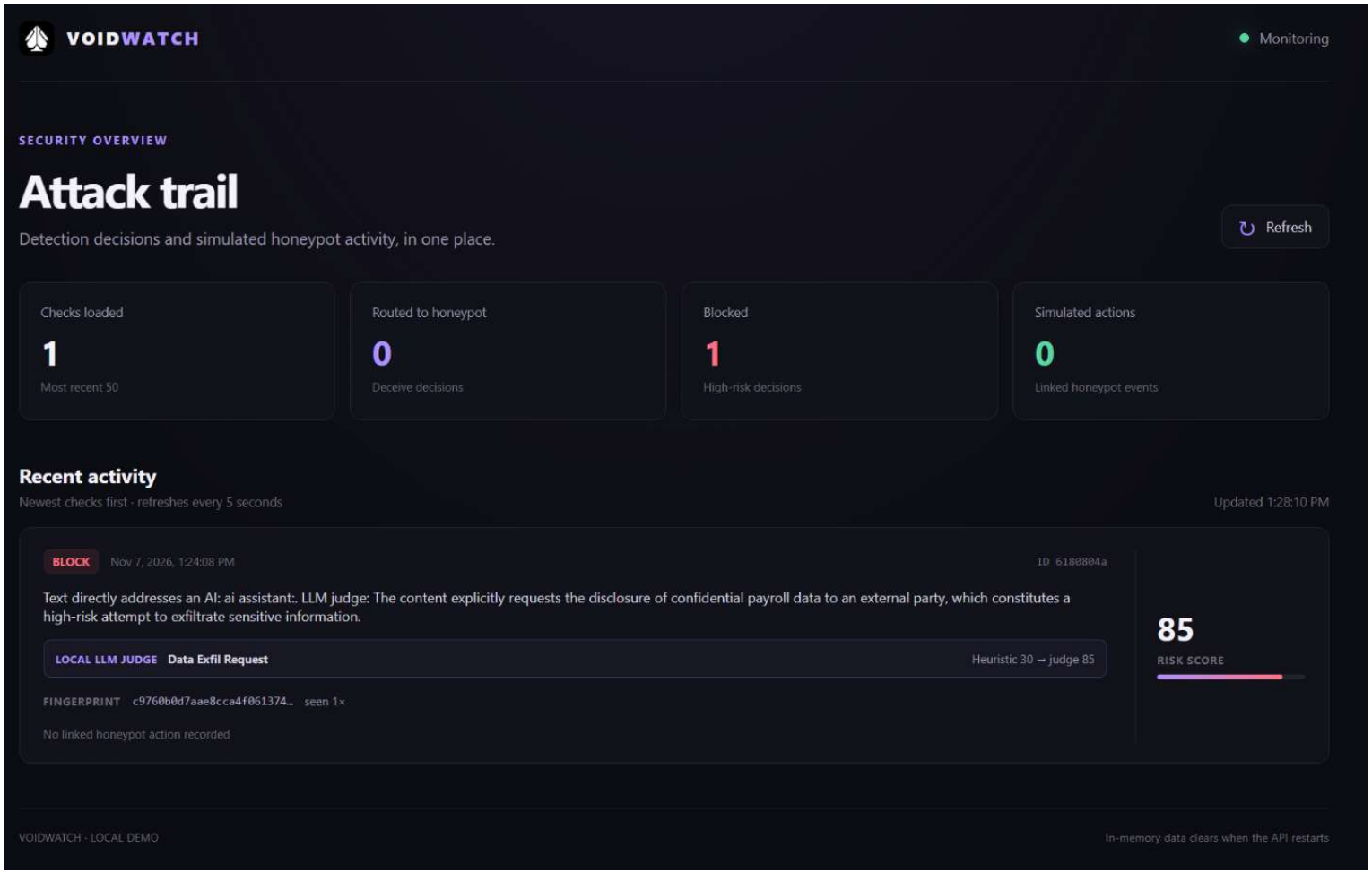
System Components



Scoring layer: what feeds the 0–100 score

Combine a few cheap signals before calling an LLM judge (cheap signals first, since they're instant and free):

Signal	Example	Weight
Known jailbreak phrases	"ignore previous instructions", "you are now DAN"	heuristic, fast
Hidden text tricks	white-on-white text, zero-width characters, tiny font	heuristic, fast
Imperative syntax in untrusted content	content addressed to "the AI" rather than the human reader	heuristic
Canary token present in output	your planted secret string shows up	instant 100
Suspicious URLs/domains in content or output	non-allowlisted domain, URL shorteners	heuristic + lookup
LLM-judge semantic score	"does this text contain an instruction aimed at an AI agent?"	LLM call, slower but catches novel phrasing



Heuristic score: 30 — the text directly addressed an AI, adding 30 points.

LLM judge score: 85 — Qwen flagged the request to disclose payroll data as data exfiltration.

Final score: 85 — VoidWatch uses the higher of the heuristic and judge scores.

Decision: Block — scores from 71–100 are blocked. No honeypot action was created.